

Fundamentals

(Data Types, Character Strings, Formatted IO, Operators, Expressions and Statements)

Mao

Outline



- **Keywords:** `int` , `short` , `long` , `unsigned` , `char` , `float` , `double` , `Bool` , `Complex` , `Imaginary`
- **Operator:** `sizeof`
- **Function:** `scanf()`
- The basic data types that C uses
- The distinctions between integer types and floating-point types
- Writing constants and declaring variables of those types
- How to use the `printf()` and `scanf()` functions to read and write values of different types

Outline



- Function: `strlen()`
- Keywords: `const`
- Character strings
- How character strings are created and stored
- How you can use `scanf()` and `printf()` to read and display character strings
- How to use the `strlen()` function to measure string lengths
- The C preprocessor's `#define directive` and ANSI C's `const` modifier for creating symbolic constants

Outline



- **Operators: = - * /**
- **% ++ -- (type)**
- **C's multitudinous operators, including those used for common arithmetic operations**
- **Operator precedence and the meanings of the terms statement and expression**

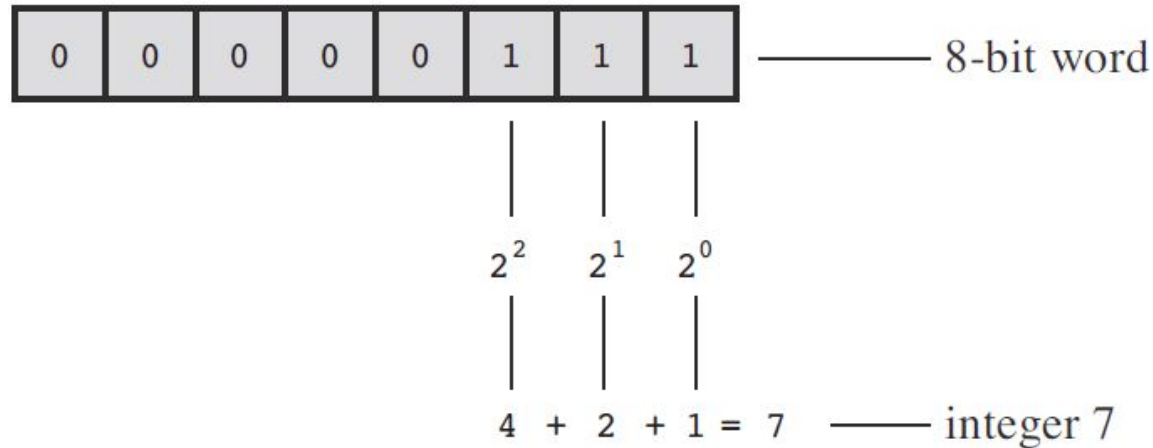
Table 3.1 C Data Keywords

Original K&R Keywords	C90 K&R Keywords	C99 Keywords
int	signed	_Bool
long	void	_Complex
short		_Imaginary
unsigned		
char		
float		
double		

Take a look at the fundamental type keywords recognized by C. K&R C recognized seven keywords relating to types. The C90 standard added two to the list. The C99 standard adds yet another three.

Integer Versus Floating-Point Types

An integer is a number with no fractional part. In C, an integer is never written with a decimal point.



Storing the integer 7 using a binary code.

Integer Versus Floating-Point Types



A floating-point number more or less corresponds to what mathematicians call a real number . Real numbers include the numbers between the integers. Some floating-point numbers are 2.75, 3.16E7, 7.00, and $2e-8$.

Type	Definition	Control Character	Limits
SIGNED TYPES			
int	Integer		-2147483648 to 2147483647
short	Short Integer		-32768 to 32767
long	Long Integer	l or L	-2147483648 to 2147483647
float	Floating Decimal Number	f or F	1.17549e-038 to 3.40282e+038
double	Double Decimal Number		2.22507e-308 to 1.79769e+308
long double	Long Decimal Number		2.22507e-308 to 1.79769e+308
char	Character		-128 to 127
UNSIGNED TYPES			
unsigned int	Unsigned Integer		0 to 4294967295
unsigned short	Unsigned Short Integer		0 to 65535
unsigned long	Unsigned Long Integer		0 to 4294967295
unsigned char	Unsigned Character		0 to 255
OTHER			
bool	True or False		True = 1 and False = 0

Listing 3.2 The print1.c Program

```
/* print1.c-displays some properties of printf() */
#include <stdio.h>
int main(void)
{
    int ten = 10;
    int two = 2;

    printf("Doing it right: ");
    printf("%d minus %d is %d\n", ten, 2, ten - two );

    printf("Doing it wrong: ");
    printf("%d minus %d is %d\n", ten ); // forgot 2 arguments

    return 0;
}
```

You can use the `printf()` function to print int types. The `%d` is called a format specifier because it indicates the form that `printf()` uses to display a value. Each `%d` in the format string must be matched by a corresponding int value in the list of items to be printed.

Printing int Values

You may have to insert a `getchar();` statement in the code for some IDEs to keep the program execution window from closing immediately.

Compiling and running the program produced this output on one system:

```
Doing it right: 10 minus 2 is 8
```

```
Doing it wrong: 10 minus 16 is 1650287143
```

To display an integer in octal notation instead of decimal, use `%o` instead of `%d` . To display an integer in hexadecimal, use `%x` . If you want to display the C prefixes, you can use specifiers `%#o` , `%#x` , and `%#X` to generate the `0` , `0x` , and `0X` prefixes respectively.

Listing 3.3 The `bases.c` Program

```
/* bases.c--prints 100 in decimal, octal, and hex */
#include <stdio.h>
int main(void)
{
    int x = 100;

    printf("dec = %d; octal = %o; hex = %x\n", x, x, x);
    printf("dec = %d; octal = %#o; hex = %#x\n", x, x, x);

    return 0;
}
```

Compiling and running this program produces this output:

```
dec = 100; octal = 144; hex = 64
dec = 100; octal = 0144; hex = 0x64
```

Declaring Other Integer Types



Other integer types are declared in the same manner as the int type.

```
long int estine;
```

```
long johns;
```

```
short int erns;
```

```
short ribs;
```

```
unsigned int s_count;
```

```
unsigned long headcount;
```

```
unsigned short yesvotes;
```

Using Characters: Type char



The `char` type is used for storing characters such as letters and punctuation marks, but **technically it is an integer type**. Why? Because the `char` type actually stores integers, not characters.

To handle characters, the computer uses a numerical code in which certain integers represent certain characters. The most commonly used code in the U.S. is the **ASCII code**.

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	>	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

<https://commons.wikimedia.org/wiki/File:ASCII-Table-wide.pdf>

Using Characters: Type char

Declaring Type char Variables:

```
char response;
```

```
char itable, latan;
```

This code would create three char variables: response , itable , and latan .

You can assign the character A to grade with the following initialization:

```
char grade = 'A';
```

Using Characters: Type char

A single character contained **between single quotes** is a C character constant . When the compiler sees 'A' , it converts the 'A' to the proper code value. **The single quotes are essential.** Here's another example:

```
char broiled; /* declare a char variable */
```

```
broiled = 'T'; /* OK */
```

```
broiled = T; /* NO! Thinks T is a variable */
```

```
broiled = "T"; /* NO! Thinks "T" is a string */
```

Using Characters: Type char

Because characters are really stored as numeric values, you can also use the numerical code to assign values:

```
char grade = 65; /* ok for ASCII, but poor style */
```

In this example, 65 is type int , but, because the value is smaller than the maximum char size, it can be assigned to grade without any problems.

Because 65 is the ASCII code for the letter A .

To represent certain awkward characters in C is to use special symbol sequences. These are called escape sequences .

Table 3.2 Escape Sequences

Sequence	Meaning
\a	Alert (ANSI C).
\b	Backspace.
\f	Form feed.
\n	Newline.
\r	Carriage return.
\t	Horizontal tab.
\v	Vertical tab.
\\	Backslash (\).
\'	Single quote (').
\"	Double quote (").
\?	Question mark (?).
\0oo	Octal value. (o represents an octal digit.)
\xhh	Hexadecimal value. (h represents a hexadecimal digit.)

Printing Characters



The `printf()` function uses `%c` to indicate that a character should be printed. Recall that a character variable is stored as a 1-byte integer value.

Therefore, if you print the value of a char variable with the usual `%d` specifier, **you get an integer**. The `%c` format specifier tells `printf()` to **display the character that has that integer as its code value**.

Listing 3.5 The charcode.c Program

```
/* charcode.c-displays code number for a character */
#include <stdio.h>
int main(void)
{
    char ch;

    printf("Please enter a character.\n");
    scanf("%c", &ch);    /* user inputs character */
    printf("The code for %c is %d.\n", ch, ch);

    return 0;
}
```

Here is a sample run:

```
Please enter a character.
C
The code for C is 67.
```

Printing Characters



The `scanf()` function then fetches the character you typed, and the ampersand (`&`) causes the character to be assigned to the variable `ch` . The `printf()` function then prints the value of `ch` twice, first as a character (prompted by the `%c` code) and then as a decimal integer (prompted by the `%d` code).

Declaring Floating-Point Variables

Floating-point variables are declared and initialized in the same manner as their integer cousins. Here are some examples:

```
float noah, jonah;
```

```
double trouble = 3.1415;
```

```
float planck = 6.63e-34;
```

```
long double gnp;
```

```
int num;
```

Number	Scientific Notation	Exponential Notation
1,000,000,000	= 1.0×10^9	= 1.0e9
123,000	= 1.23×10^5	= 1.23e5
322.56	= 3.2256×10^2	= 3.2256e2
0.000056	= 5.6×10^{-5}	= 5.6e-5

The `printf()` function uses the `%f` format specifier to print type float and double numbers using decimal notation, and it uses `%e` to print them in exponential notation.

If your system supports the hexadecimal format for floating-point numbers, you can use `a` or `A` instead of `e` or `E`. The long double type requires the `%Lf`, `%Le`, and `%La` specifiers to print that type. Note that both float and double use the `%f`, `%e`, or `%a` specifier for output.

Listing 3.7 The `showf_pt.c` Program

```
/* showf_pt.c -- displays float value in two ways */
#include <stdio.h>
int main(void)
{
    float aboat = 32000.0;
    double abet = 2.14e9;
    long double dip = 5.32e-5;

    printf("%f can be written %e\n", aboat, aboat);
    // next line requires C99 or later compliance
    printf("And it's %a in hexadecimal, powers of 2 notation\n", aboat);

    printf("%f can be written %e\n", abet, abet);
    printf("%Lf can be written %Le\n", dip, dip);

    return 0;
}
```

This is the output, provided your compiler is C99/C11 compliant:

```
32000.000000 can be written 3.200000e+04
And it's 0x1.f4p+14 in hexadecimal, powers of 2 notation
2140000000.000000 can be written 2.140000e+09
0.000053 can be written 5.320000e-05
```

Complex and Imaginary Types



Many computations in science and engineering use complex and imaginary numbers.

In brief, there are three complex types, called `float _Complex` , `double _Complex` , and `long double _Complex`. A `float _Complex` variable, for example, would contain two float values, one representing the real part of a complex number and one representing the imaginary part.

Similarly, there are three imaginary types, called `float _Imaginary`, `double _Imaginary`, and `long double _Imaginary`.

Type Sizes

What type sizes does your system use? Use `sizeof()`.

Listing 3.8 The `typesize.c` Program

```
/* typesize.c -- prints out type sizes */
#include <stdio.h>
int main(void)
{
    /* c99 provides a %zd specifier for sizes */
    printf("Type int has a size of %zd bytes.\n", sizeof(int));
    printf("Type char has a size of %zd bytes.\n", sizeof(char));
    printf("Type long has a size of %zd bytes.\n", sizeof(long));
    printf("Type long long has a size of %zd bytes.\n",
           sizeof(long long));
    printf("Type double has a size of %zd bytes.\n",
           sizeof(double));
    printf("Type long double has a size of %zd bytes.\n",
           sizeof(long double));
    return 0;
}
```

Listing 4.1 The talkback.c Program

```
// talkback.c -- nosy, informative program
#include <stdio.h>
#include <string.h>      // for strlen() prototype
#define DENSITY 62.4    // human density in lbs per cu ft
int main()
{
    float weight, volume;
    int size, letters;
    char name[40];       // name is an array of 40 chars

    printf("Hi! What's your first name?\n");
    scanf("%s", name);
    printf("%s, what's your weight in pounds?\n", name);
    scanf("%f", &weight);
    size = sizeof name;
    letters = strlen(name);
    volume = weight / DENSITY;
    printf("Well, %s, your volume is %2.2f cubic feet.\n",
           name, volume);
    printf("Also, your first name has %d letters,\n",
           letters);
    printf("and we have %d bytes to store it.\n", size);

    return 0;
}
```

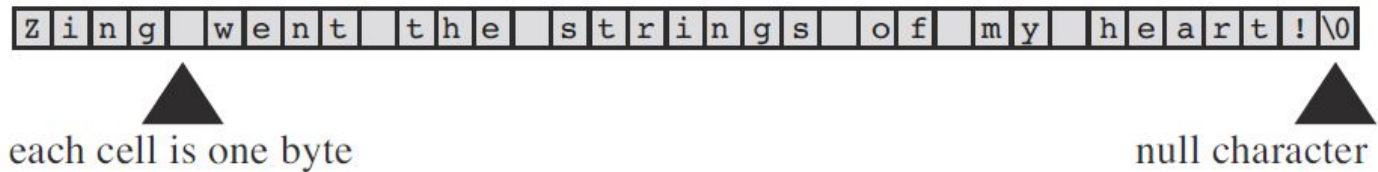
Running talkback.c produces results such as the following:

```
Hi! What's your first name?
Christine
Christine, what's your weight in pounds?
154
Well, Christine, your volume is 2.47 cubic feet.
Also, your first name has 9 letters,
and we have 40 bytes to store it.
```

Here are the main new features of this program:

- It uses an array to hold a character string . Here, someone's name is read into the array, which, in this case, is a series of 40 consecutive bytes in memory, each able to hold a single character value.
- It uses the `%s` conversion specification to handle the input and output of the string. Note that `name` , unlike `weight` , does not use the `&` prefix when used with `scanf()` . (As you'll see later, both `&weight` and `name` are addresses.)
- It uses the C preprocessor to define the symbolic constant `DENSITY` to represent the value 62.4 .

Type char Arrays and the Null Character

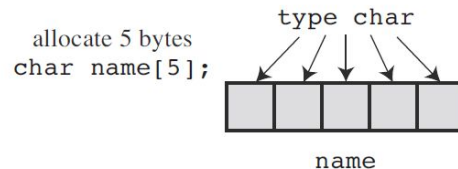
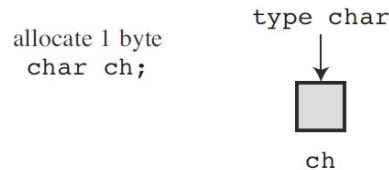


C has no special variable type for strings. Instead, **strings are stored in an array of type char.** Characters in a string are stored in adjacent memory cells, one character per cell, and an array consists of adjacent memory locations.

Note that the character **\0** in the last array position. This is the **null character**, and C uses it to mark the end of a string. The null character is not the digit zero.

Type char Arrays and the Null Character

The brackets after name identify it as an array. The 40 within the brackets indicates the number of elements in the array. The **char** identifies the type of each element.



Declaring a variable versus declaring an array.

There is no failure just changeable code.

Type char Arrays and the Null Character

String literals **can be assigned without size**. Here, the name of the string `str` acts as a pointer because it is an array.

```
char str[] = "snow";
```

String literals can be assigned with a predefined size. **But we should always account for one extra space which will be assigned to the null character**. If we want to store a string of size `n` then we should always declare a string with a size equal to or greater than **`n+1`**.

```
char str[5] = "snow";
```

```
char str[5] = {'s', 'n', 'o', 'w', '\0'}; // NULL character '\0' is required in this declaration
```

Type char Arrays and the Null Character

```
char str[5] = "snow";
```

≠

```
char str[5];
```

```
str = "snow"; // Error
```

Using Strings

You do not have to put the null character into the **name** array yourself. That task is done for you by **scanf()** when it reads the input. Nor do you include a null character in the character string constant **PRAISE**.

Listing 4.2 The praise1.c Program

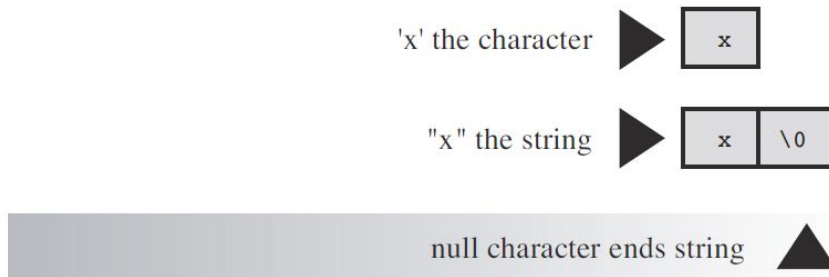
```
/* praise1.c -- uses an assortment of strings */
#include <stdio.h>
#define PRAISE "You are an extraordinary being."
int main(void)
{
    char name[40];

    printf("What's your name? ");
    scanf("%s", name);
    printf("Hello, %s. %s\n", name, PRAISE);

    return 0;
}
```

Strings Versus Characters

The string constant `"x"` is not the same as the character constant `'x'`. One difference is that `'x'` is a basic type (`char`), but `"x"` is a derived type, an array of `char`. A second difference is that `"x"` really consists of two characters, `'x'` and `'\0'`, the **null** character.



The character 'x' and the string "x".

Listing 4.3 The praise2.c Program

```
/* praise2.c */
// try the %u or %lu specifiers if your implementation
// does not recognize the %zd specifier
#include <stdio.h>
#include <string.h>      /* provides strlen() prototype */
#define PRAISE "You are an extraordinary being."
int main(void)
{
    char name[40];

    printf("What's your name? ");
    scanf("%s", name);
    printf("Hello, %s. %s\n", name, PRAISE);
    printf("Your name of %zd letters occupies %zd memory cells.\n",
           strlen(name), sizeof name);
    printf("The phrase of praise has %zd letters ",
           strlen(PRAISE));
    printf("and occupies %zd memory cells.\n", sizeof PRAISE);

    return 0;
}
```

What's your name? Serendipity Chance

Hello, Serendipity. You are an extraordinary being.

Your name of 11 letters occupies 40 memory cells.

The phrase of praise has 31 letters and occupies 32 memory cells.

The strlen() Function

The array name has 40 memory cells, and that is what the `sizeof` operator reports. **Only the first 11 cells are needed to hold Serendipity**, however, and that is what `strlen()` reports.

The twelfth cell in the array name contains the null character, and its presence tells `strlen()` when to stop counting.



11 characters.

Constants and the C Preprocessor

First comes **#define** . Next comes the symbolic **name** for the constant and then the **value** for the constant. (Note that this construction does not use the = sign.) So the general

#define NAME value

Note that no semicolon is used.

Listing 4.4 The pizza.c Program

```
/* pizza.c -- uses defined constants in a pizza context */
#include <stdio.h>
#define PI 3.14159
int main(void)
{
    float area, circum, radius;

    printf("What is the radius of your pizza?\n");
    scanf("%f", &radius);
    area = PI * radius * radius;
    circum = 2.0 * PI * radius;
    printf("Your basic pizza parameters are as follows:\n");
    printf("circumference = %1.2f, area = %1.2f\n", circum,
        area);
    return 0;
}
```

Constants and the C Preprocessor

The `#define` statement can be used for **character and string constants**, too. Just use single quotes for the former and double quotes for the latter. The following examples are valid:

```
#define BEEP '\a'
```

```
#define TEE 'T'
```

```
#define ESC '\033'
```

```
#define OOPS "Now you have done it!"
```

The const Modifier

C90 added a second way to create symbolic constants—using the **const** keyword to convert a declaration for a variable into a declaration for a constant:

```
const int MONTHS = 12; // MONTHS a symbolic constant for 12
```

This makes MONTHS into a **read-only** value.

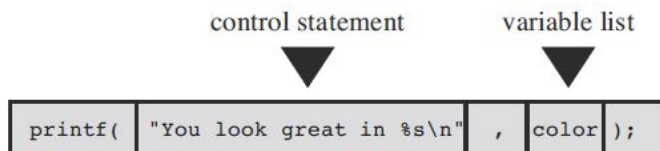
That is, you can display **MONTHS** and use it in calculations, but you cannot alter the value of **MONTHS** .

Table 4.3 Conversion Specifiers and the Resulting Printed Output

Conversion	Output Specification
%a	Floating-point number, hexadecimal digits and p-notation (C99/C11).
%A	Floating-point number, hexadecimal digits and P-notation (C99/C11).
%c	Single character.
%d	Signed decimal integer.
%e	Floating-point number, e-notation.
%E	Floating-point number, e-notation.
%f	Floating-point number, decimal notation.
%g	Use %f or %e, depending on the value. The %e style is used if the exponent is less than -4 or greater than or equal to the precision.
%G	Use %f or %E, depending on the value. The %E style is used if the exponent is less than -4 or greater than or equal to the precision.
%i	Signed decimal integer (same as %d).
%o	Unsigned octal integer.
%p	A pointer.
%s	Character string.
%u	Unsigned decimal integer.
%x	Unsigned hexadecimal integer, using hex digits 0f.
%X	Unsigned hexadecimal integer, using hex digits 0F.
%%	Prints a percent sign.

Using printf()

Control-string is the phrase enclosed in double quotes.



Listing 4.6 The `printout.c` Program

```
/* printout.c -- uses conversion specifiers */
#include <stdio.h>
#define PI 3.141593
int main(void)
{
    int number = 7;
    float pies = 12.75;
    int cost = 7800;

    printf("The %d contestants ate %f berry pies.\n", number,
           pies);
    printf("The value of pi is %f.\n", PI);
    printf("Farewell! thou art too dear for my possessing,\n");
    printf("%c%d\n", '$', 2 * cost);

    return 0;
}
```

The output, of course, is

```
The 7 contestants ate 12.750000 berry pies.
The value of pi is 3.141593.
Farewell! thou art too dear for my possessing,
$15600
```

Conversion Specification Modifiers for printf()

Table 4.4 The `printf()` Modifiers

Modifier	Meaning
flag	The five flags (<code>-</code> , <code>+</code> , space, <code>#</code> , and <code>0</code>) are described in Table 4.5. Zero or more flags may be present. Example: <code>"%-10d"</code> .
digit(s)	The minimum field width. A wider field will be used if the printed number or string won't fit in the field. Example: <code>"%4d"</code> .
<i>.digit(s)</i>	Precision. For <code>%e</code> , <code>%E</code> , and <code>%f</code> conversions, the number of digits to be printed to the right of the decimal. For <code>%g</code> and <code>%G</code> conversions, the maximum number of significant digits. For <code>%s</code> conversions, the maximum number of characters to be printed. For integer conversions, the minimum number of digits to appear; leading zeros are used if necessary to meet this minimum. Using only <code>.</code> implies a following zero, so <code>%.f</code> is the same as <code>%.0f</code> . Example: <code>"%5.2f"</code> prints a <code>float</code> in a field five characters wide with two digits after the decimal point.

Listing 4.8 The floats.c Program

```
// floats.c -- some floating-point combinations
#include <stdio.h>

int main(void)
{
    const double RENT = 3852.99; // const-style constant

    printf("%f\n", RENT);
    printf("%e\n", RENT);
    printf("%4.2f\n", RENT);
    printf("%3.1f\n", RENT);
    printf("%10.3f\n", RENT);
    printf("%10.3E\n", RENT);
    printf("%+4.2f\n", RENT);
    printf("%010.2f\n", RENT);

    return 0;
}
```

This time, the program uses the keyword `const` to create a symbolic constant. The output is

```
*3852.990000*
*3.852990e+03*
*3852.99*
*3853.0*
* 3852.990*
* 3.853E+03*
*+3852.99*
*0003852.99*
```

Using scanf()

Like `printf()` , `scanf()` uses a control string followed by a list of arguments. The control string indicates the destination data types for the input stream of characters.

Listing 4.15 The input.c Program

```
// input.c -- when to use &
#include <stdio.h>
int main(void)
{
    int age;           // variable
    float assets;      // variable
    char pet[30];      // string

    printf("Enter your age, assets, and favorite pet.\n");
    scanf("%d %f", &age, &assets); // use the & here
    scanf("%s", pet);           // no & for char array
    printf("%d $%.2f %s\n", age, assets, pet);

    return 0;
}
```

Here is a sample exchange:

```
Enter your age, assets, and favorite pet.
38
92360.88 llama
38 $92360.88 llama
```

Assignment Operator: =

In C, the equal sign does not mean “equals.” Rather, it is a value-assigning operator. The statement

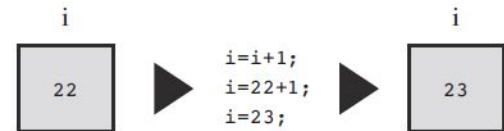
```
bmw = 2002;
```

assigns the value **2002** to the variable named **bmw**.

Consider the following common type of computer statement:

```
i = i + 1;
```

It means “Find the value of the variable named **i**, add 1 to that value, and then assign this new value to the variable **i**”



+ and -

The addition operator **+** causes the two values on either side of it to be added together. For example, the statement

```
printf("%d", 4 + 20);
```

causes **the number 24 to be printed**, not the expression **4 + 20**.

The subtraction operator **-** causes the number after the **-** sign to be subtracted from the number before the sign. The statement

```
takehome = 224.00 - 24.00;
```

assigns the value **200.0** to **takehome** .

More Operators

Name	Symbol	Description	Example
Addition	+	Adds two operands	int a = 5, b = 2; int c = a+b; // c = 7
Subtraction	-	Subtracts second operand from the first	int a = 5, b = 2; int c = a-b; // c = 3
Multiplication	*	Multiplies two operands	int a = 5, b = 2; int c = a*b; // c = 10
Division	/	Divides first operand by the second operand	int a = 5, b = 2; int c = a/b; // c = 2
Modulo	%	Returns the remainder an integer division	int a = 5, b = 2; int c = a%b; // c = 1

++ and --

right++	x = 10	y = x++	x = 11 y = 10
++left	x = 10	y = ++x	x = 11 y = 11
right--	x = 10	y = x--	x = 9 y = 10
--left	x = 10	y = --x	x = 9 y = 9

++ and --

Listing 5.11 The post_pre.c Program

```
/* post_pre.c -- postfix vs prefix */
#include <stdio.h>
int main(void)
{
    int a = 1, b = 1;
    int a_post, pre_b;

    a_post = a++; // value of a++ during assignment phase
    pre_b = ++b;   // value of ++b during assignment phase
    printf("a  a_post  b  pre_b \n");
    printf("%1d %5d %5d %5d\n", a, a_post, b, pre_b);

    return 0;
}
```

`+=`, `-=`, `*=`, `/=` and `%=`

Operator	Meaning	Arithmetic Operation	Abbreviated Expression
<code>+=</code>	Collect and Assign	<code>a=a+b</code>	<code>a+=b</code>
<code>-=</code>	Subtract and Assign	<code>a=a-b</code>	<code>a-=b</code>
<code>*=</code>	Multiply and Assign	<code>a=a*b</code>	<code>a*=b</code>
<code>/=</code>	Divide and Assign	<code>a=a/b</code>	<code>a/=b</code>
<code>%=</code>	Get and Assign Mod	<code>a=a%b</code>	<code>a%=b</code>

Reference



- **Stephen Prata. C Primer Plus (6th Edition). Addison-Wesley Professional, 2013.**